Session 3 : Virtualization & Hypervisors

RECAP: Session 2 : Deployment & Service Models

1. Cloud Deployment Models: Public cloud; private cloud: on-premise vs hosted; hybrid & cloud bursting; community cloud
2. Multi-Cloud: Motivations: resilience, negotiation & regulation; operational cost of multiple platforms
3. Service Models: IaaS, PaaS & SaaS: what each provides & abstracts; representative services
4. Beyond the Big Three: Function as a Service (FaaS) & serverless; AI/ML-as-a-Service; responsibility boundaries across models
5. Cloud Economics & Providers: CapEx vs OpEx; total cost of ownership (TCO); pricing: on-demand, committed use & spot; AWS, Azure, GCP & Indian providers

Agenda for Today's Session

1. Virtualization Basics: Physical vs virtual resources; consolidation, isolation & utilization; types: server, storage, network, desktop, application
2. Hypervisors: Type 1 (bare-metal) vs Type 2 (hosted); Virtual Machine Monitor (VMM) & its role
3. Virtualization Techniques: Full virtualization; para-virtualization; hardware-assisted virtualization (VT-x / AMD-V)
4. Virtual Machine Anatomy: vCPU, virtual memory, virtual disk & virtual NIC; machine images, templates & snapshots
5. VM Lifecycle & Migration: Provision, start, suspend, snapshot, clone & terminate; live migration & its uses

---------- SECTION 1: 1. Virtualization Basics: Physical vs virtual resources; consolidation, isolation & utilization; types: server, storage, network, desktop, application ----------

Slide 1: Virtualization: The Big Picture

Heading: Virtualization Landscape

Subheading: Physical To Virtual

Content:

Virtualization uses software to create virtual versions of computing resources, allowing physical hardware to be shared efficiently while keeping workloads isolated.

- Server: Multiple virtual machines run on one physical server.
- Storage:Virtual storage is created from shared physical disks.
- Network: Virtual networks are created over shared physical network hardware.
- Desktop: Virtual desktops can be accessed remotely from different devices.
- Application: Applications can run in isolated virtual environments.

A **hypervisor** — the software layer that creates and manages VMs — sits between the physical hardware and each virtual resource

Visual Guidance: Diagram: physical server block at bottom, hypervisor layer above it, five branches on top labelled Server, Storage, Network, Desktop, Application, each leading to a virtual icon.

Speaker Notes: "Emphasise that virtualization applies one abstraction principle to five different resource categories, not just computation. Interviewers often ask for all five types in order, so list server, storage, network, desktop and application together."

Slide 2: Why Virtualization Pays Off

Heading: Three Core Benefits

Subheading: Mechanism And Effect

Content:

Before virtualization, each physical server typically ran one operating system and one application, leaving most capacity idle.

Benefit	How Achieved	Effect
Consolidation	Multiple VMs share one physical host	Fewer physical servers needed for same workload
Isolation	Hypervisor enforces separate memory and CPU per VM	A fault in one VM cannot affect others
Utilization	Idle CPU/memory cycles reassigned across VMs on demand	Average server utilization rises sharply versus one-app-per-server

Visual Guidance: Small bar icon beside the table contrasting low utilization before virtualization with high utilization after.

Speaker Notes: "Note that hypervisor-enforced isolation is what stops a crashed or compromised VM from affecting others sharing the host. Interviewers frequently ask why virtualization raises utilization — tie the answer to dynamic reallocation of idle cycles."

Slide 3: Server Virtualization Deep Dive

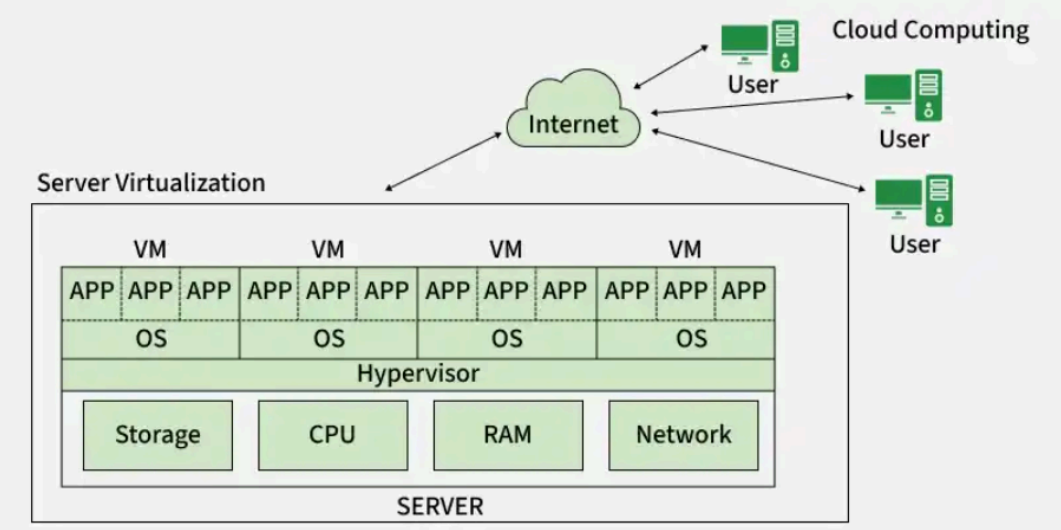
Heading: Server Virtualization

Subheading: Full OS Per VM

Content:

Server virtualization means running multiple virtual servers on a single physical server, so its CPU, memory, and storage are shared instead of sitting idle. Each virtual server is called a virtual machine (VM) and works like an independent computer.

- Introduced to end one-app-per-server hardware waste
- Guest OS is fully independent of host OS
- Common use: dev/test environments, legacy app hosting
- AWS example: Amazon EC2 (Elastic Compute Cloud) launches VM instances



Visual Guidance: Reuse the physical-to-hypervisor-to-VMs diagram from the overview, zoomed into one VM box showing a full guest OS inside it.

Speaker Notes: "Server virtualization is the type already met through the hypervisor, now framed formally as one of five categories. Interview note: the guest OS, not just the application, is what gets virtualized here."

Slide 4: Storage Virtualization Explained

Heading: Storage Virtualization

Subheading: Pooled Virtual Volumes

Content:

Storage virtualization pools capacity from multiple physical disks into logical volumes, letting applications see storage independent of the hardware beneath.

- Applications do not need to know which disk holds their data
- Volumes can be resized without moving data manually
- Snapshots: point-in-time copies for quick recovery
- Replication: copies kept in another location for safety
- AWS example: Amazon EBS provides virtual disks for EC2

Analogy: A bank combines money from multiple branches into a shared pool that customers can access from anywhere. Similarly, ***storage virtualization combines capacity from multiple physical disks into a single logical storage pool that applications can use without needing to know where the data is physically stored.**

Visual Guidance: Diagram: several physical disk icons feeding into a single pooled cylinder labelled Virtual Volume, with an application drawing from the cylinder.

Speaker Notes: "Storage virtualization lets a volume resize or migrate without touching the physical array. Contrast with RAID, which protects fixed disks rather than pooling them."

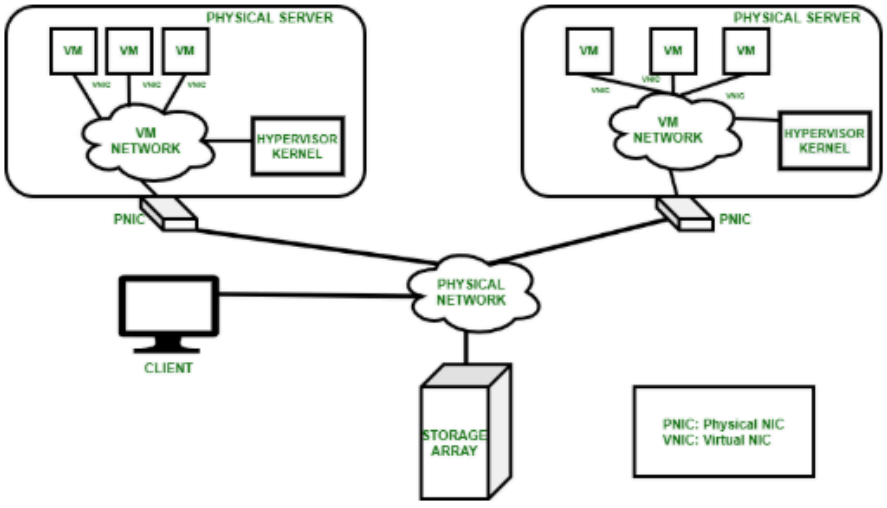
Slide 5: Network Virtualization Explained

Heading: Network Virtualization

Subheading: Logical Nets Over Physical

Content:

Network virtualization uses software to create multiple separate virtual networks on top of one shared physical network. Each virtual network has its own addressing and settings.



Network Virtualization In VDC

- Virtual switches replace physical wiring changes
- Overlay networks tunnel traffic across shared links
- Isolation prevents one tenant's traffic reaching another's
- AWS example: Amazon VPC (Virtual Private Cloud) isolated network

Analogy : A building can use one shared electricity connection while separate meters track each tenant's usage. Similarly, **network virtualization uses one physical network to create multiple separate logical networks, each with its own addressing and access rules.**

Visual Guidance: Diagram: one physical network cable branching into several coloured logical network loops, each labelled with its own address range.

Speaker Notes: "Network virtualization is the basis of software-defined networking (SDN), separating logical topology from physical cabling. A VPC is a fully isolated virtual network, unlike a VLAN which only segments one physical LAN."

Slide 6: Desktop And App Virtualization

Heading: Desktop & App Virtualization

Subheading: Remote Access Delivery

Content:

Desktop virtualization delivers a full desktop from a remote host; application virtualization streams or isolates without local installation.

- Desktop: full OS session streamed remotely
- Application: only the app runs remotely

- Both remove dependency on local hardware
- AWS examples: Amazon WorkSpaces desktops, AppStream 2.0 apps

Analogy: Cable TV delivers a channel without a studio at home — just as desktop and app virtualization deliver a desktop or an app from a remote host without local installation.

Visual Guidance: Diagram: a remote data-centre icon streaming two outputs to a local screen — one labelled Full Desktop, one labelled Single App.

Speaker Notes: "Desktop virtualization, or virtual desktop infrastructure (VDI), delivers a whole session, while application virtualization streams just one app — a common interview mix-up. Both remove local installation and local hardware dependence."

Slide 7: Comparing The Five Types

Heading: Virtualization Types Compared

Subheading: Side-By-Side Summary

Content:

Each virtualization type abstracts a different physical resource, but all follow the same pattern of separating logical use from physical ownership.

Type	What Is Virtualized	Real AWS Example
Server	Physical machine into VMs	Amazon EC2
Storage	Physical disks into volumes	Amazon EBS
Network	Physical wiring into virtual nets	Amazon VPC
Desktop	Physical workstation into remote session	Amazon WorkSpaces
Application	remotely streamed or isolated app	AWS AppStream 2.0

Visual Guidance: Full-width table as the sole visual; no additional diagram needed on this slide.

Speaker Notes: "This table maps each type to its resource and a real AWS example, useful for fast recall. Only server virtualization runs a full guest OS; the other four abstract a resource without running one."

---------- SECTION 2: 2. Hypervisors: Type 1 (bare-metal) vs Type 2 (hosted); Virtual Machine Monitor (VMM) & its role ----------

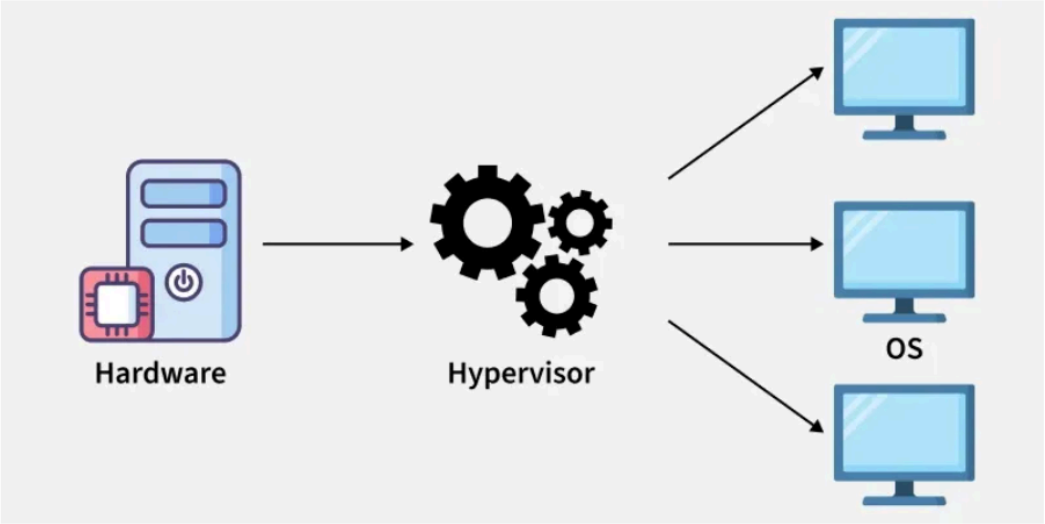
Slide 8: Virtual Machine Monitor: Role Explained

Heading: Hypervisor And VMM

Subheading: Role And Function

Content:

A hypervisor — also called a Virtual Machine Monitor (VMM) — is a thin layer of software that sits between physical hardware and one or more virtual machines (VMs). A VM is a software-based emulation of a physical computer, running its own guest OS and applications. The hypervisor creates each VM and controls how it accesses the underlying physical hardware.



What does a Hypervisor do?

- Creates and manages VMs
- Allocates CPU, memory, storage, and network resources
- Controls how VMs access physical hardware
- Keeps VMs isolated from one another

Analogy: An apartment building's management office allocates each tenant a unit, meters their power and water separately, and keeps one tenant's plumbing problem from affecting

another's — just as a hypervisor allocates CPU, memory and device access to each VM while keeping them isolated from one another.

Visual Guidance: Single diagram: Hardware base, VMM layer above it, three VM boxes on top each with its own OS, arrows from VMM to each resource type.

Speaker Notes: "A VMM virtualizes CPU, memory and I/O without any VM noticing another's presence. Interviewers commonly ask exactly what a VMM virtualizes."

Slide 9: Type 1 Hypervisor: Bare-Metal Architecture

Heading: Type 1 Bare-Metal

Subheading: Runs On Hardware

Content:

A Type 1 hypervisor runs directly on physical hardware, with no host operating system beneath it, and behaves as a thin operating system itself.

- Examples: VMware ESXi, Microsoft Hyper-V, KVM
- AWS EC2 runs on the Nitro hypervisor (KVM-based)
- Direct hardware access gives near-native performance
- Standard choice for production data centres and cloud platforms
- Requires dedicated server hardware, not a desktop OS

Visual Guidance: Stack diagram, bottom to top: Physical Hardware, Hypervisor (VMM), then VM boxes directly above it — no host OS layer shown.

Speaker Notes: "Type 1 hypervisors power most production cloud data centres today. Interviewers often ask why Type 1 outperforms Type 2 — the missing host OS layer is the answer."

Slide 10: Type 2 Hypervisor: Hosted Architecture

Heading: Type 2 Hosted

Subheading: Runs On Host OS

Content:

A Type 2 hypervisor runs as an application on top of an existing host operating system, relying on it for device drivers and scheduling.

- Examples: Oracle VirtualBox, VMware Workstation, Parallels Desktop

- Every I/O call passes through host OS first
- Extra translation layer adds performance overhead
- Simple to install, common for desktops and testing

Visual Guidance: Stack diagram, bottom to top: Physical Hardware, Host OS, Hypervisor application, then VM boxes on top.

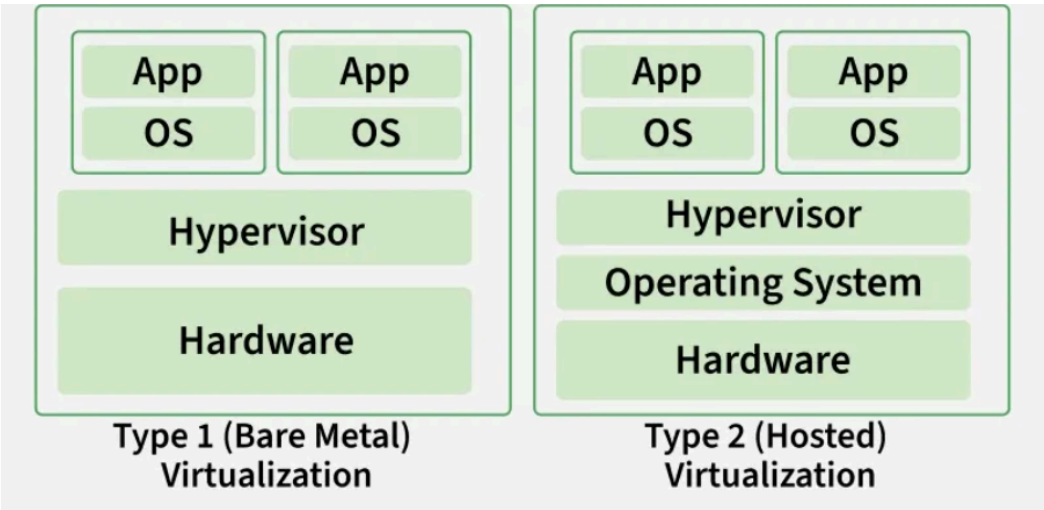
Speaker Notes: "This is the kind of hypervisor used for local desktop virtualization and quick testing setups. A frequent interview question asks for one real product example of each hypervisor type."

Slide 11: Type 1 Vs Type 2: Trade-Offs

Heading: Bare-Metal Vs Hosted

Subheading: Stack And Trade-Offs

Content:



Types of Hypervisor

Both types embed a VMM to virtualize hardware; they differ in what layer sits between that VMM and the physical machine.

Aspect	Type 1 (Bare-Metal)	Type 2 (Hosted)
Host layer	None — runs on hardware	Full host OS underneath
Performance	Near-native, low overhead	Added translation overhead
Examples	VMware ESXi, Hyper-V, KVM	VirtualBox, VMware Workstation
Typical use	Data centres, cloud platforms	Desktop, dev and test

Visual Guidance: Side-by-side stack diagrams: Type 1 (Hardware→Hypervisor→VMs) next to Type 2 (Hardware→Host OS→Hypervisor App→VMs) for direct visual contrast.

Speaker Notes: "Type selection matters most for latency-sensitive workloads rather than casual testing. Interviewers often ask which hypervisor type clouds run in production, and why that choice performs better."

---------- SECTION 3: 3. Virtualization Techniques: Full virtualization; para-virtualization; hardware-assisted virtualization (VT-x / AMD-V) ----------

Slide 12: Three Techniques For Virtualizing The CPU

Heading: Techniques Landscape

Subheading: Three Core Approaches

Content:

A guest OS expects to run privileged code directly on hardware, so the hypervisor must intercept or replace those operations. The three techniques differ in how:

- Full virtualization: unmodified guest; software handles sensitive instructions (trap-and-emulate plus binary translation)
- Para-virtualization: modified guest, direct hypercalls to hypervisor
- Hardware-assisted: CPU itself traps sensitive instructions

Visual Guidance: Three side-by-side boxes labelled Full, Para-virtualization, Hardware-Assisted, each with an arrow style (software trap / hypercall / CPU trap) from guest to hypervisor.

Speaker Notes: "Each technique differs only in where the trap-and-handle work happens — software, a modified guest, or hardware. Interviewers frequently ask to name and contrast all three, so anchor this map before the detail."

Slide 13: Full Virtualization: Trap And Translate

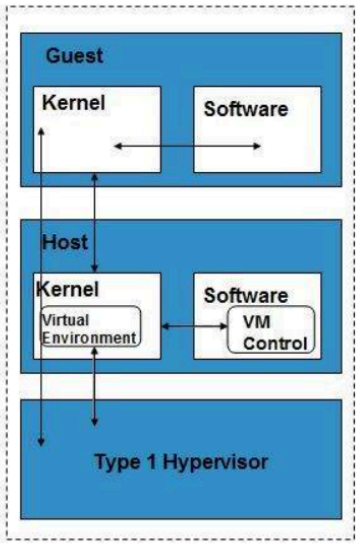
Heading: Full Virtualization

Subheading: Binary Translation Trap

Content:

Full virtualization runs an unmodified guest OS. On early x86 CPUs, some sensitive instructions did not trap when executed. The hypervisor therefore used binary translation:

it scans and rewrites guest kernel code on the fly, while normal user-mode code runs directly on the CPU.



- Instructions that do trap are handled by trap-and-emulate
- Sensitive instructions that don't trap are rewritten by binary translation
- The guest needs no source or kernel changes
- Overhead falls mainly on kernel-heavy, privileged operations
- Used by early VMware ESX and Workstation

Analogy: A hotel guest uses the same key card to access their room without knowing how the hotel's locking system works behind the scenes. Similarly, ***full virtualization gives an unmodified guest OS a familiar virtual hardware environment, while the hypervisor handles access to the actual physical hardware**

Visual Guidance: Guest OS box issuing an instruction, arrow labelled trap-plus-translate into the hypervisor, then a second arrow into the physical CPU.

Speaker Notes: "Full virtualization needs zero guest modification, but the translation step costs CPU cycles on every trap. Interviewers often ask why early VMware products ran slower than native execution — this trap-and-emulate overhead is the answer."

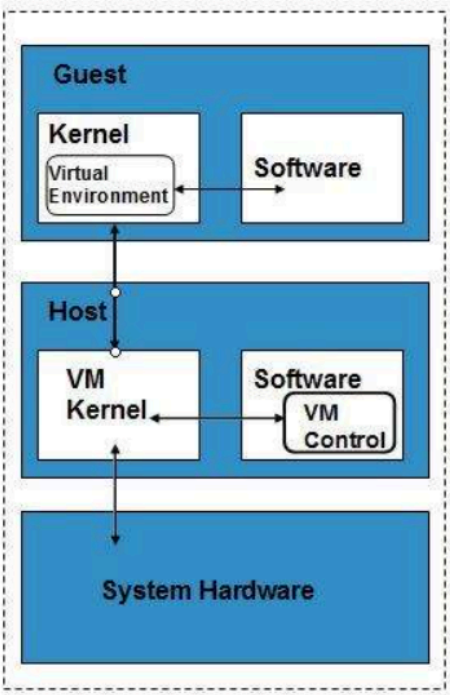
Slide 14: Para-Virtualization: Guest-Aware Hypercalls

Heading: Para-Virtualization

Subheading: Guest-Aware Hypercalls

Content:

In para-virtualization, the guest operating system is modified to work with the hypervisor directly instead of behaving as if it has complete control of the hardware.



- Hypercalls replace trapped instructions, skipping translation
- Requires a modified guest kernel and paravirtualized drivers
- Lower overhead than full virtualization for I/O-heavy guests
- Para-virtual drivers (virtio, VMware Tools) are still used inside modern hardware-assisted VMs for fast I/O
- Example: Xen paravirtualized (PV) guest mode

Analogy: In a restaurant, instead of waiting for a waiter to notice that you need something, you can use a service bell to directly request assistance from the staff. Similarly, in ***para-virtualization***, the guest OS knows it is running in a virtual environment and uses ***hypercalls to directly request services from the hypervisor***

Visual Guidance: Modified guest OS box with a direct solid arrow labelled hypercall into the hypervisor, contrasted with a dotted trap arrow reused from the full-virtualization diagram.

Speaker Notes: "Para-virtualization trades guest-OS portability for lower overhead, since the kernel must be patched to issue hypercalls. Interview angle: this is why Xen PV guests needed a modified Linux kernel while hardware-assisted guests do not."

Slide 15: Hardware-Assisted Virtualization And Comparison

Heading: Hardware-Assisted Virtualization

Subheading: CPU-Level Trapping

Content:

Hardware-assisted virtualization means the CPU has built-in features (Intel VT-x, AMD-V) that help the hypervisor run VMs faster and more safely.

- **CPU support:** The hypervisor controls the VM, while the guest OS runs directly on the CPU.
- **CPU intervention:** When the guest OS needs hypervisor control, the CPU pauses the VM and hands control back to the hypervisor.
- **Memory support:** EPT (Intel) and NPT (AMD) help the hardware translate VM memory addresses.
- **Used by:** KVM, VMware ESXi, and Hyper-V.
- **BIOS/UEFI setting:** CPU virtualization must be enabled for many VM tools to work.

Technique	Guest OS Change	Trap Mechanism	Overhead
Full (binary translation)	None	Software translation	Highest for kernel-heavy work
Para-virtualization	Modified kernel	Guest-initiated hypercalls	Low, especially for I/O
Hardware-assisted	None	CPU VM exit / entry	Low. Modern default, usually combined with para-virtual I/O drivers

Visual Guidance: CPU diagram showing root mode (hypervisor) below non-root mode (guest), plus the comparison table beneath it.

Speaker Notes: "Hardware-assisted virtualization is what makes modern cloud-scale hypervisors practical, since VT-x/AMD-V let unmodified guests run at near-native speed. A common interview question asks why hypervisors moved away from pure binary translation — this table is the answer."

---------- SECTION 4: 4. Virtual Machine Anatomy: vCPU, virtual memory, virtual disk & virtual NIC; machine images, templates & snapshots ----------

Slide 16: Anatomy of a Virtual Machine

Heading: VM Anatomy Map

Subheading: Hardware And Packaging

Content:

A VM presents four virtualized hardware components to its guest OS, while a separate set of artifacts package and manage the VM itself.

- vCPU: virtual processor core scheduled onto physical cores
- Virtual memory: guest RAM carved from host physical memory
- Virtual disk: file-backed storage acting as a physical drive
- Virtual NIC: software network adapter for guest connectivity
- Images, templates, snapshots: how VMs are packaged and reused

Visual Guidance: Diagram: physical machine parts (CPU, RAM, disk, NIC) on left, mapped through the hypervisor to four matching virtual parts inside one VM box on right.

Speaker Notes: "Point out that these four resources are exactly what gets allocated the moment a VM is created. Exams commonly ask for the components of a VM — this map is the checklist."

Slide 17: vCPU and Virtual Memory

Heading: vCPU And Memory

Subheading: Compute Allocation

Content:

A vCPU is a logical processor the hypervisor time-slices onto physical cores; virtual memory is the guest's RAM window mapped to host physical pages.

- Total vCPUs across all VMs on a host can exceed its physical cores (oversubscription)
- Oversubscription risks contention when guests demand cycles simultaneously
- Memory overcommit achieved via ballooning or host-side swapping
- Swapping under overcommit degrades guest performance sharply

Visual Guidance: Split diagram: physical cores and RAM on left, vCPUs and virtual memory blocks on right, with mapping arrows and a small scheduler icon in between.

Speaker Notes: "Contrast vCPU count against physical core count to show oversubscription is normal, not an error. Interview question: can a VM have more vCPUs than the host has cores — yes, scheduling handles it."

Slide 18: Virtual Disk and Virtual NIC

Heading: Disk And NIC

Subheading: Storage And Network

Content:

A virtual disk is a single host file the guest treats as a raw physical drive; a virtual NIC is a software adapter wired to a virtual switch.

- Common virtual disk formats: VMDK, VHD/VHDX, QCOW2, VDI (VirtualBox native)
- Disk modes: fixed-size pre-allocated or dynamically growing
- Virtual NIC is assigned its own virtual MAC address
- Virtual switch or bridge connects vNIC to the physical network. Common vNIC modes: NAT, bridged, host-only.

Visual Guidance: Diagram: VM box with a disk icon linked to a file on host storage, and a NIC icon linked through a virtual switch to the physical network card.

Speaker Notes: "Note that a virtual disk is just a file, which is why it can be copied or moved like any other file. Expect a question distinguishing fixed-size from dynamically-expanding disk formats."

Slide 19: Images, Templates and Snapshots

Heading: VM Packaging Artifacts

Subheading: Three Reuse Types

Content:

Machine images, templates and snapshots all preserve VM state but differ in what they capture and when each is used.

Artifact	Captures	Typical Use
Machine image	OS plus base software; no runtime state	Launch new VM instances
Template	Image plus fixed hardware spec	Standardise repeated VM deployments
Snapshot	Point-in-time state of a running VM	Rollback or pre-change backup

Visual Guidance: Three-column table graphic with icons: a disk-stack icon for image, a blueprint icon for template, a camera icon for snapshot.

Speaker Notes: "Distinguish a template's fixed hardware spec from an image's reusable OS content, and note a snapshot depends on its base disk. Interview trap: a snapshot alone is not a full backup."

Slide 20: VM Creation Walkthrough

Heading: Creating A VM

Subheading: Screenshots In Sequence

Content:

Creating a VM involves choosing the operating system, allocating virtual resources, and configuring how it connects to the network.

Basic Steps

- Create the VM — Give it a name and select the guest OS and ISO image.
- Allocate resources — Assign RAM and virtual CPUs.
- Create virtual storage — Choose the virtual disk size, format, and allocation type.
- Configure networking — Select how the VM connects to the network, such as NAT.
- Start the VM — Boot the guest OS from the selected ISO image.

Important Note:

VirtualBox is used here only to build a basic understanding of how a VM is created. Creating Virtual Machines on a Cloud Platform will be covered in later sessions.

Visual Guidance: See the condensed screenshot walkthrough on the following slides — VirtualBox 7.2.8 on Windows, installing VirtualBox and creating an Ubuntu 26.04.1 LTS guest.

Speaker Notes: "VirtualBox 7.x folded memory, CPU and disk size into one screen and added a dedicated unattended-install screen — the next slides show this live."

Slide 20A: Installing Oracle VirtualBox

Heading: Installing VirtualBox

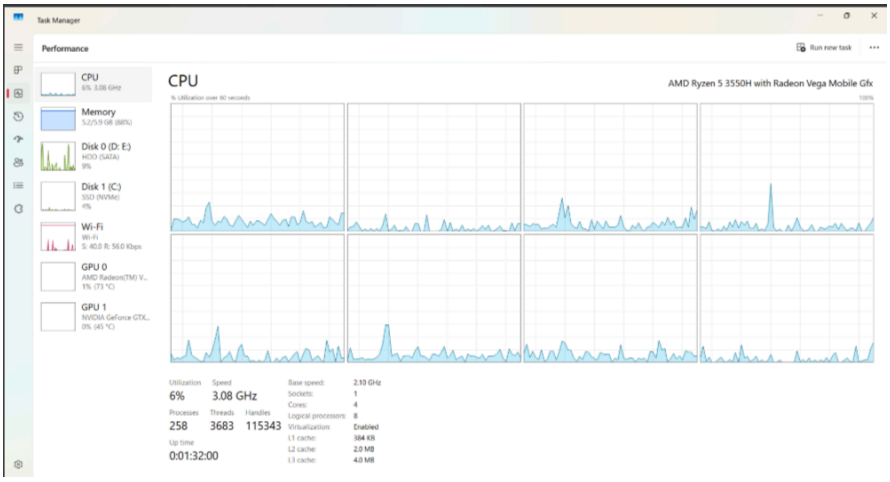
Subheading: Screenshot Walkthrough

Content:

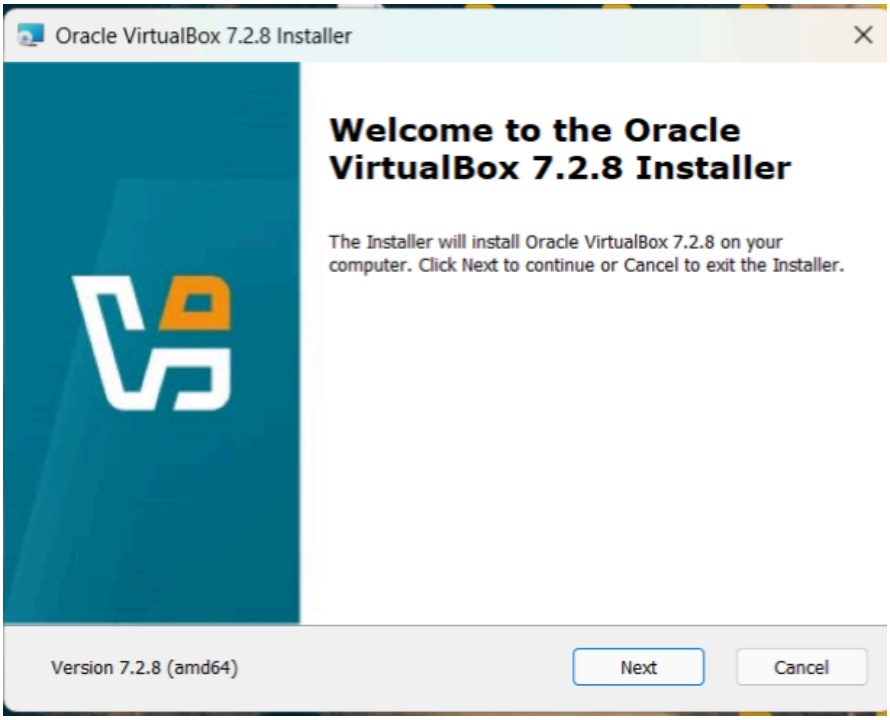
Important Note:

This walkthrough is a demonstration only. Installing and running a VM may slow down your system, so it is not mandatory to perform these steps.

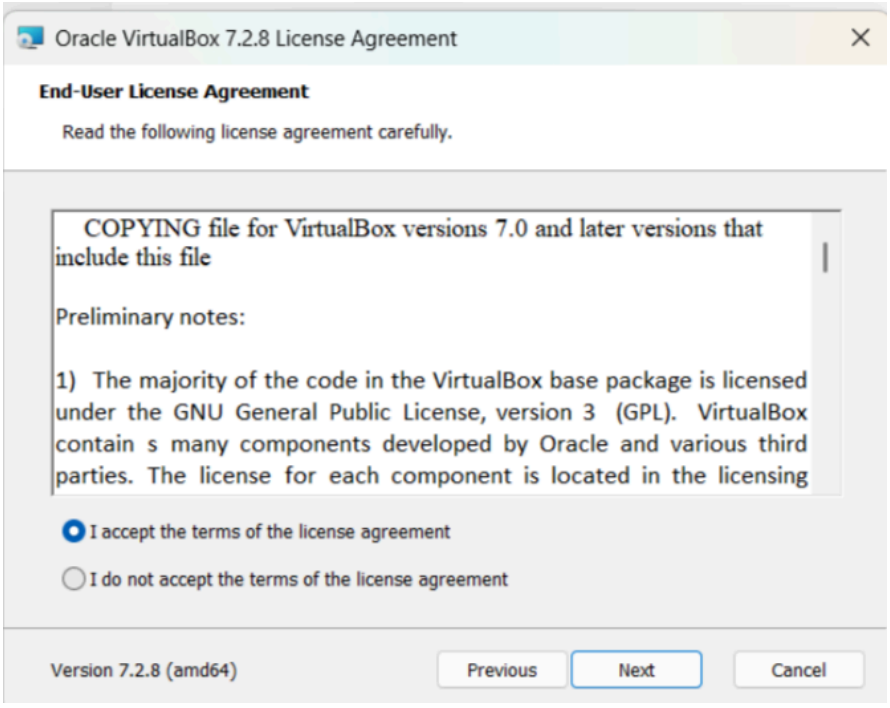
Screens from the VirtualBox 7.2.8 setup wizard, shown here for reference — trainer walks through each live rather than reading the captions below.



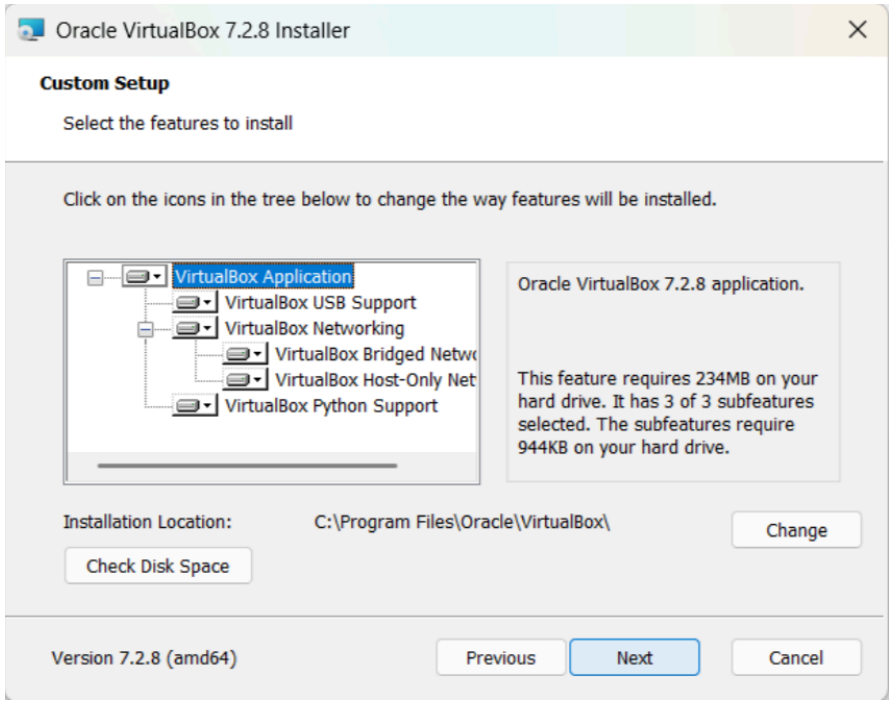
Pre-install check — Task Manager confirms CPU Virtualization: Enabled



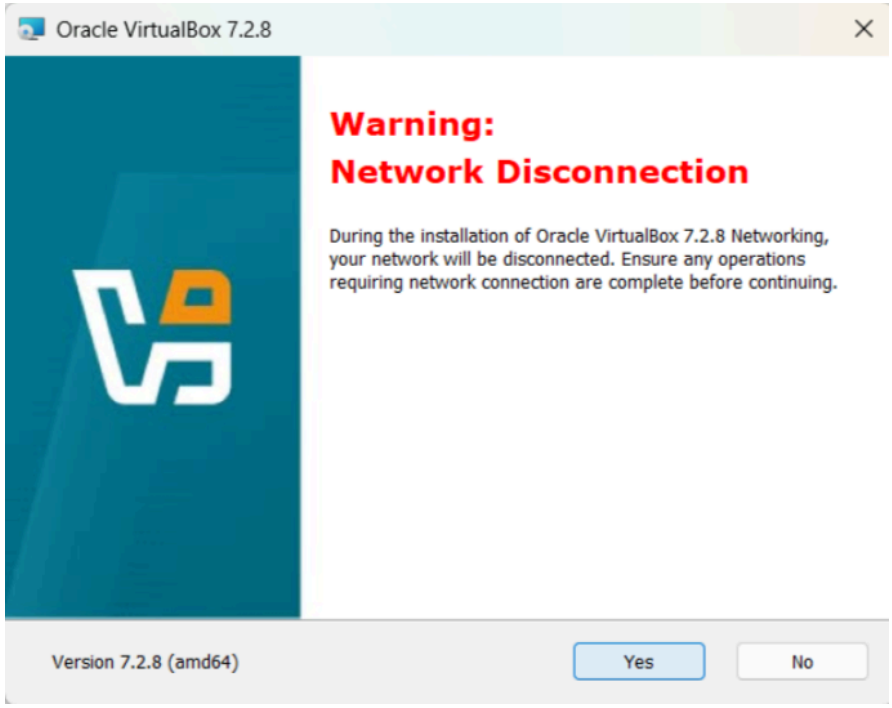
Welcome screen — confirms version 7.2.8 (amd64)



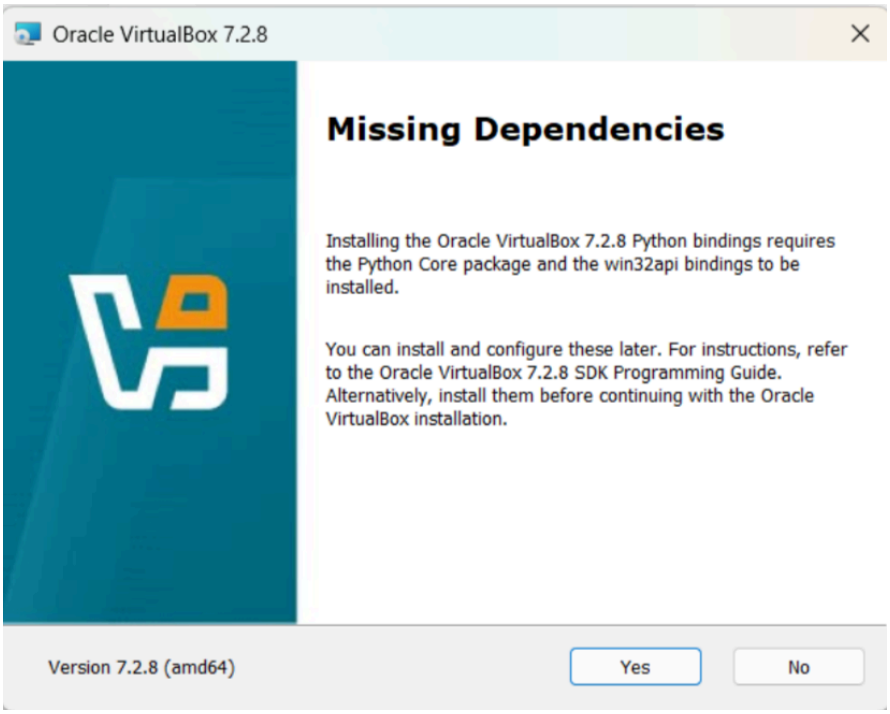
License Agreement — accept GPL v3 terms



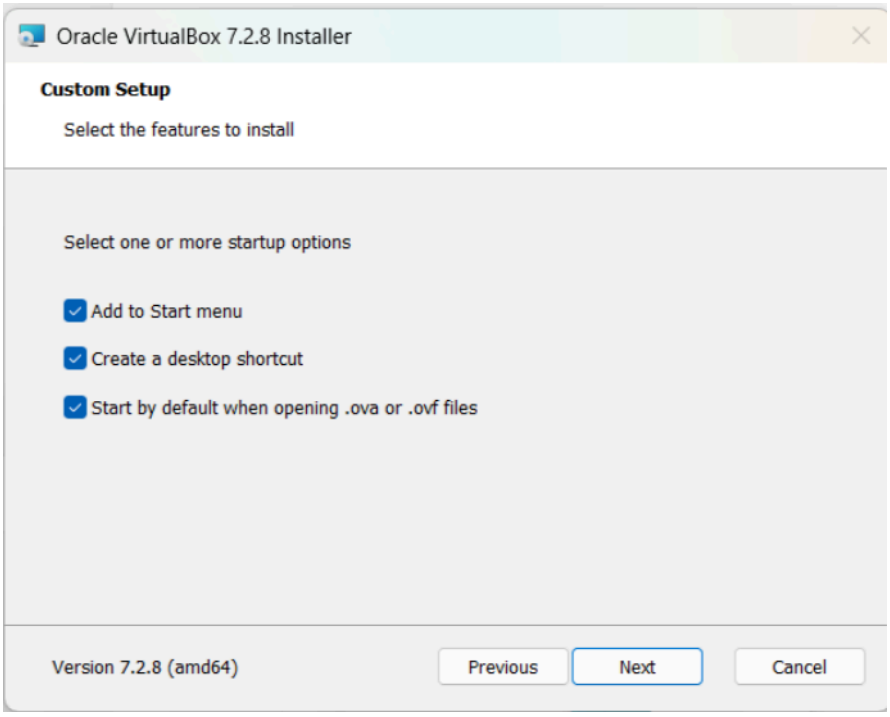
Custom Setup — keep default features



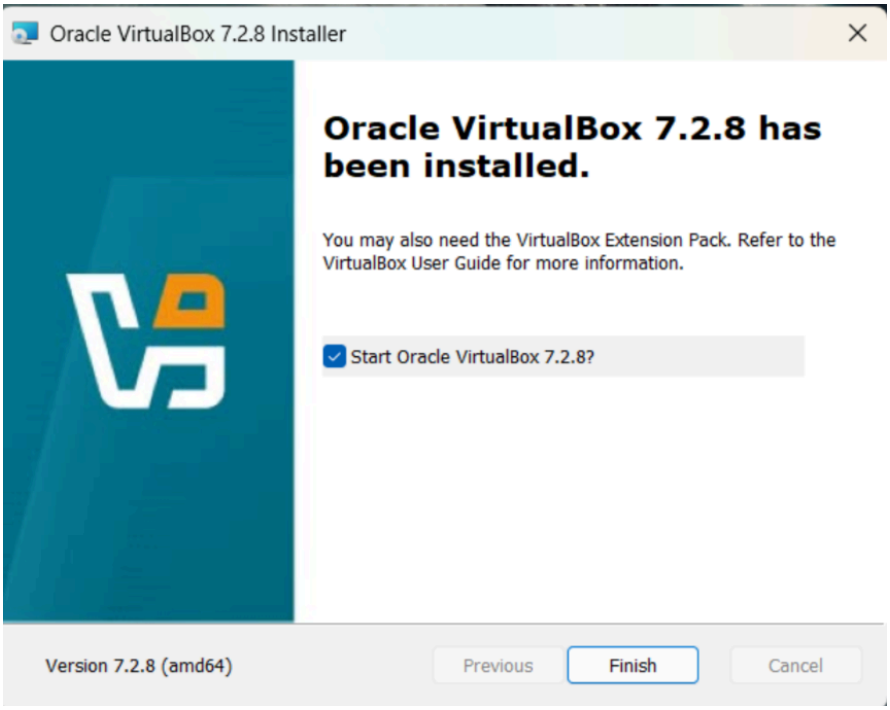
Network Disconnection warning — expected, click Yes



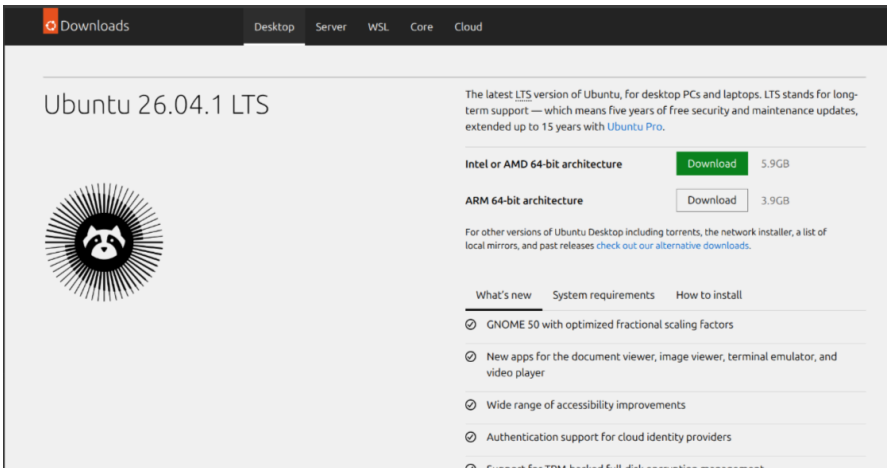
Missing Dependencies — optional, safe to skip



Startup options — keep defaults checked



Installation complete — Finish launches VirtualBox



Download guest OS ISO — Ubuntu 26.04.1 LTS Desktop (64-bit)

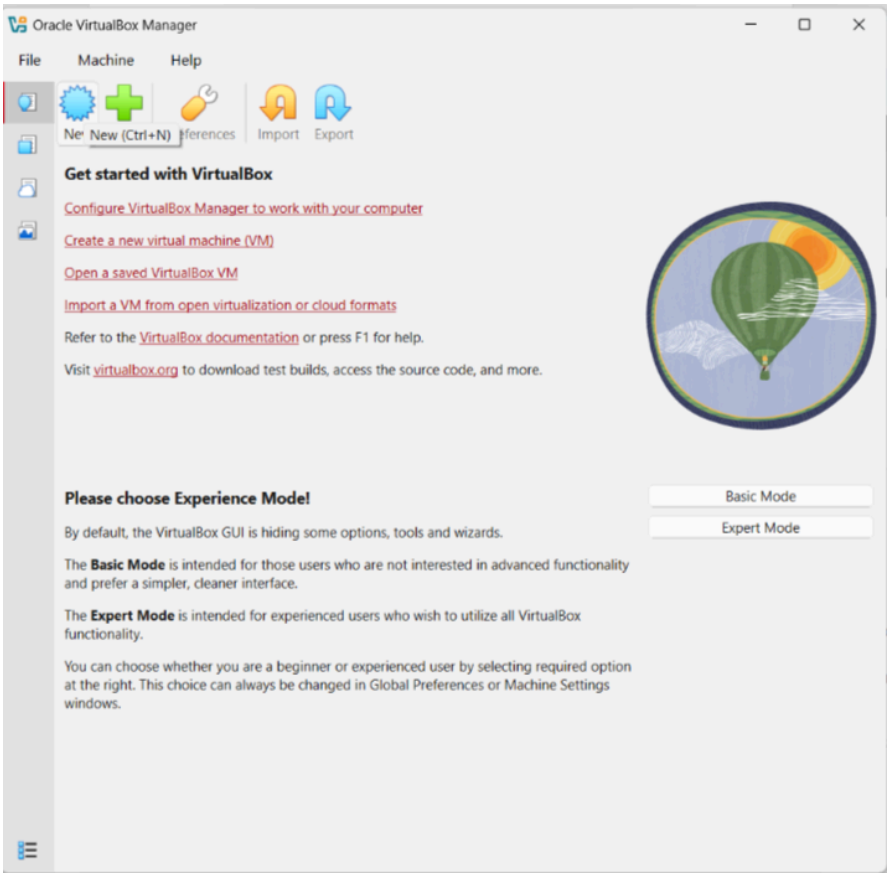
Slide 20B: Creating The VM

Heading: New Virtual Machine Wizard

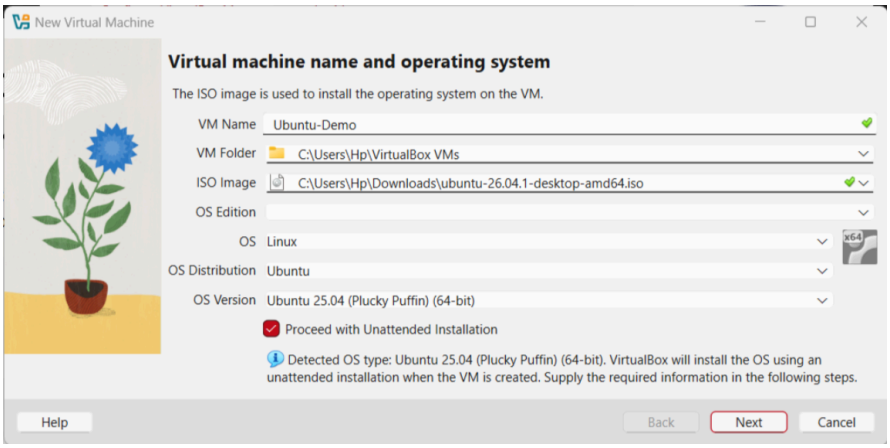
Subheading: Screenshot Walkthrough

Content:

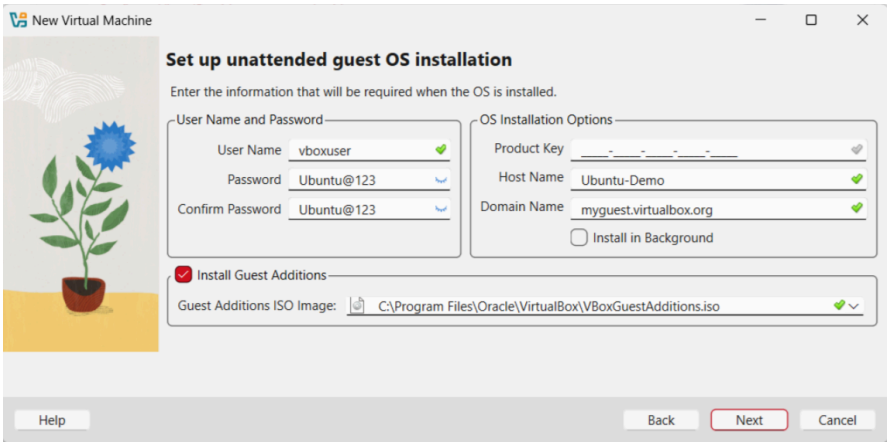
Screens from creating an Ubuntu 26.04.1 LTS VM, shown here for reference — trainer walks through each live rather than reading the captions below.



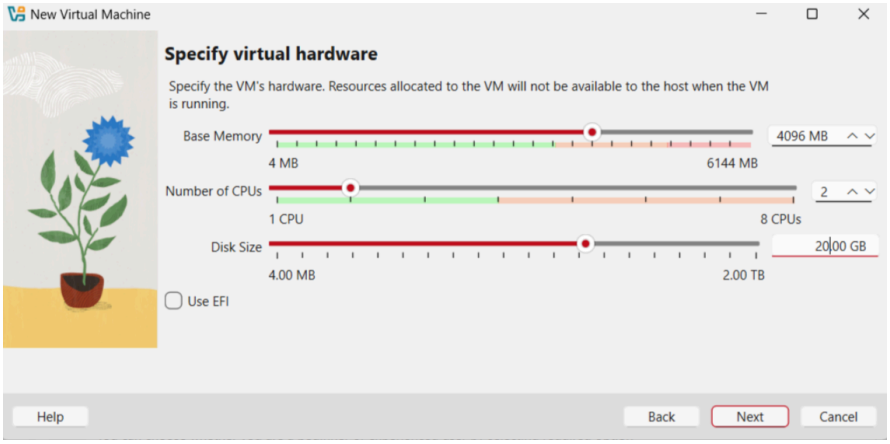
VirtualBox Manager home — click New (Ctrl+N)



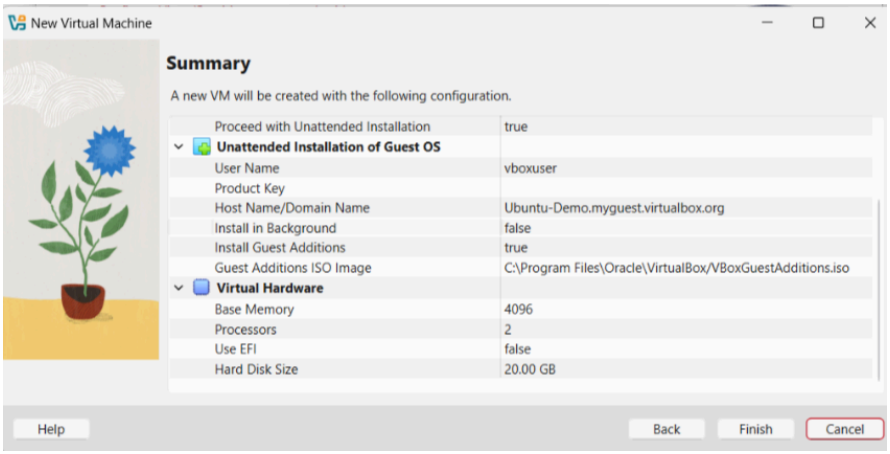
Name the VM, select ISO, confirm guest OS type



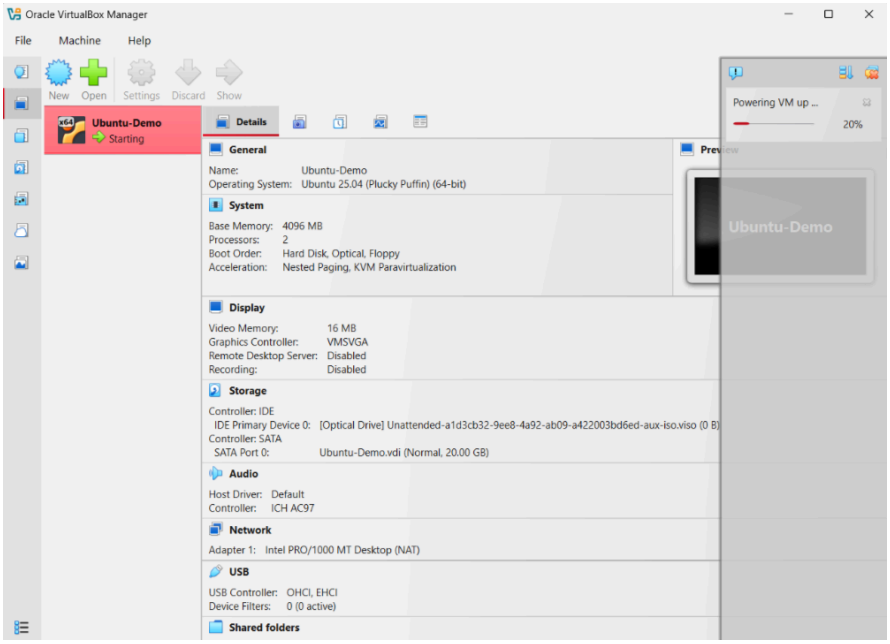
Unattended install — set username, password, hostname



Virtual hardware — memory, CPUs, disk size



Summary — Finish creates the VM



VM starting — Details pane confirms configuration

Speaker Notes: "VirtualBox auto-detected the ISO as “Ubuntu 25.04” even though it’s 26.04.1 — its OS-type list just hasn’t caught up yet; harmless."

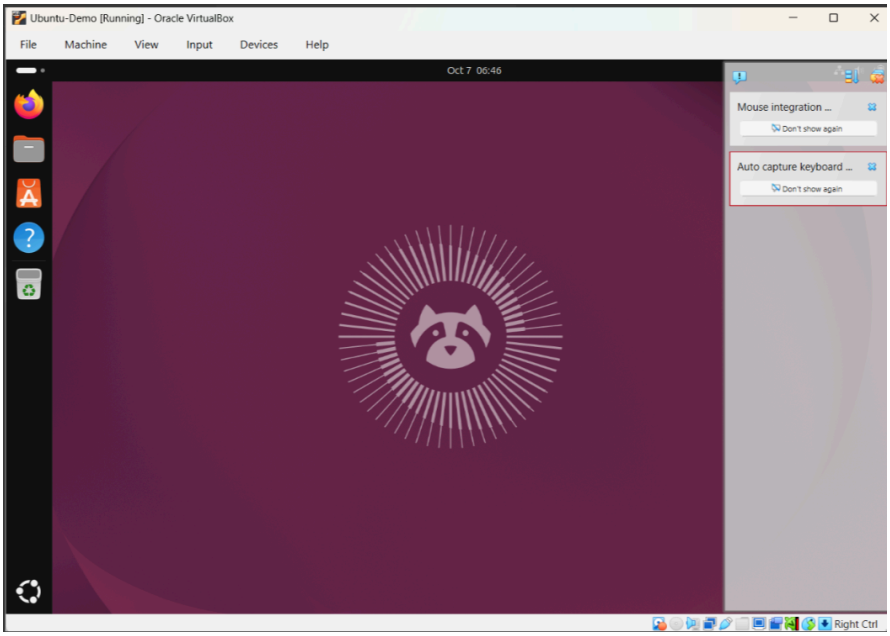
Slide 20C: VM Up And Running

Heading: First Boot

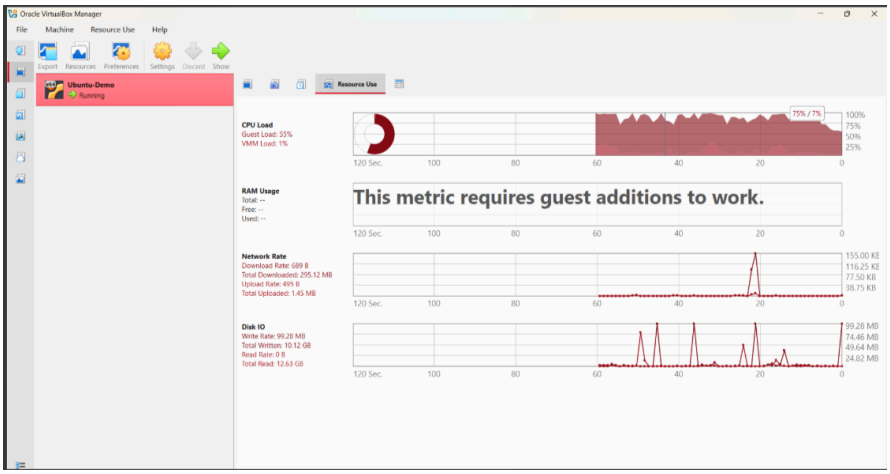
Subheading: VM Creation Confirmed

Content:

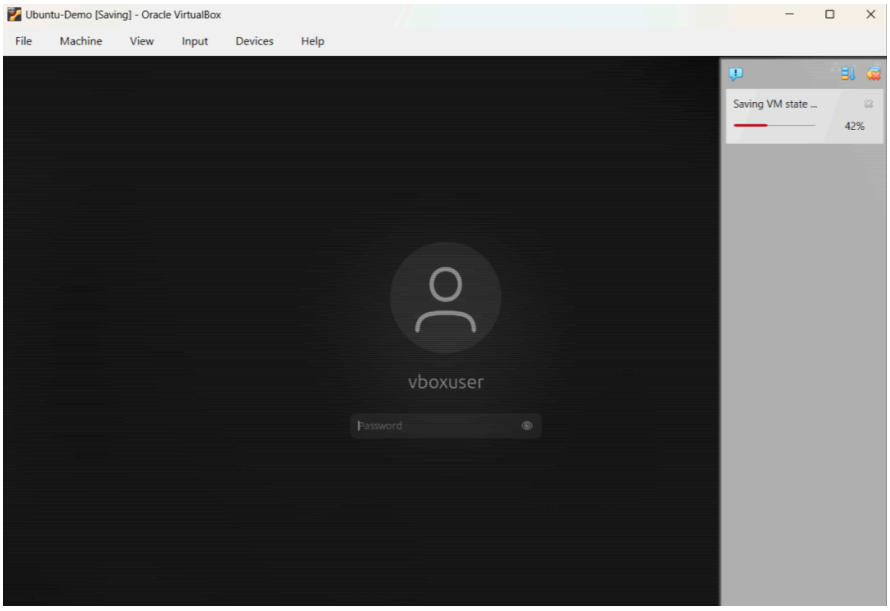
The created VM boots to its own Ubuntu desktop, confirming the walkthrough above worked end to end.



Ubuntu desktop running inside the VM



Resource Use tab — live CPU load, network rate and disk I/O



Save State — suspends the VM to disk instead of a full shutdown

---------- SECTION 5: 5. VM Lifecycle & Migration: Provision, start, suspend, snapshot, clone & terminate; live migration & its uses ----------

Slide 21: VM Lifecycle Stages Mapped

Heading: Lifecycle Stages Map

Subheading: Six Lifecycle Stages

Content:

A virtual machine moves through a defined lifecycle from creation to teardown, with administrator or platform actions triggering each transition.

Stage	What Happens
Provision	Allocate vCPU, memory, disk, NIC
Start	Boot guest OS on virtual hardware
Suspend	Pause execution; save state to disk
Snapshot	Capture point-in-time disk/state image
Clone	Duplicate VM as new independent instance
Terminate	Shut down guest; release resources

Visual Guidance: Circular flow diagram: Provision to Start, branching to Suspend/Snapshot/Clone, converging to Terminate, arrows marking transitions.

Speaker Notes: "Walk through all six stages as one continuous cycle before detailing any single one. Interviewers often ask to distinguish suspend, snapshot and clone, which comes up next."

Slide 23: Suspend Versus Snapshot Versus Clone

Heading: Suspend Snapshot Clone

Subheading: State-Capture Compared

Content:

Suspend, snapshot and clone all preserve VM state, but differ in what they capture and how the VM resumes, a frequent interview distinction.

Operation	Captures	Resumes As
Suspend	RAM and CPU state, paused to disk	Same VM, same execution point
Snapshot	Point-in-time disk (and optional memory) image	Rollback point for that VM
Clone	Full copy of disk and configuration	New, independent VM booting fresh

- AWS EC2 hibernate: stop with RAM saved to EBS
- EBS snapshot: backup and disaster-recovery point
- AMI clone: launch identical new instances at scale

Visual Guidance: Three-column icon row: pause icon for Suspend, camera icon for Snapshot, copy icon for Clone, each with a resume arrow.

Speaker Notes: "Stress that a snapshot alone does not stop the VM, while suspend does, a distinction interviewers test directly. Clone breaks the link to source state entirely, unlike the other two."

Slide 24: Live Migration Of A Running VM

Heading: Live Migration

Subheading: Moving A Running VM

Content:

Live migration transfers a running VM's memory and execution state to another physical host with near-zero downtime, unlike suspend which pauses the guest.

- Why: host maintenance, hardware-failure avoidance, load balancing
- How: memory pages copied over network while VM keeps running
- Final step: brief pause syncs last state, then resume
- Transparent to guest OS and its running applications

Analogy: A mobile call is handed from one tower to another without the conversation dropping — just as live migration moves a running VM to a new host without stopping its execution.

Visual Guidance: Diagram: Host A holding a VM, memory pages flowing across a network link to Host B, VM icon resuming on Host B.

Speaker Notes: "Contrast this with suspend, which stops the VM, since live migration never stops it. Cloud interviews often ask how providers achieve zero-downtime host maintenance, and this mechanism is the answer."

Slide 25: The Cost Of Full-OS Virtualization

Heading: Cost Of Full VMs

Subheading: Overhead Across Lifecycle

Content:

Every lifecycle stage above carries the weight of a complete guest operating system, and that weight is the practical limitation of server virtualization.

- Heavy footprint: full guest OS per VM
- Slow to start: full OS boot time
- Low density: fewer VMs fit per host
- Wastes resources at idle: full OS overhead even when idle

Visual Guidance: Icon row: heavy box for footprint, clock for boot time, sparse VM icons on a host for density, idle gauge for wasted resources.

Speaker Notes: "Tie each cost back to the lifecycle stage it slows, since heavy footprint lengthens provisioning, start-up and migration alike. All these problems come from virtualizing the whole OS, and containers only virtualize the process, which the next session covers."

Key Takeaways

1. Virtualization Basics: Physical vs virtual resources; consolidation, isolation & utilization; types: server, storage, network, desktop, application
2. Hypervisors: Type 1 (bare-metal) vs Type 2 (hosted); Virtual Machine Monitor (VMM) & its role
3. Virtualization Techniques: Full virtualization; para-virtualization; hardware-assisted virtualization (VT-x / AMD-V)
4. Virtual Machine Anatomy: vCPU, virtual memory, virtual disk & virtual NIC; machine images, templates & snapshots

5. VM Lifecycle & Migration: Provision, start, suspend, snapshot, clone & terminate; live migration & its uses

Upcoming Session : Containers vs Virtual Machines

Thank You | All the Best